

Lógica e Funcionamento dos Algoritmos

a. Bubble Sort (Ordenação por Bolha): É o algoritmo mais básico. A estratégia dele é a **troca de vizinhos**. Percorrendo a lista comparando dois elementos por vez: se o da esquerda for maior que o da direita, eles trocam de lugar. Isso faz com que o maior valor da lista vá "flutuando" até a última posição, como uma bolha. O processo se repete várias vezes até que a lista esteja totalmente ordenada e nenhuma troca seja mais necessária.

b. Selection Sort (Ordenação por Seleção): A lógica é a **seleção do menor valor**. O algoritmo olha para toda a parte bagunçada da lista, encontra qual é o menor número de todos e o troca de lugar com a primeira posição disponível. Depois, ele ignora o que está certo e repete o processo para o restante, sempre "selecionando" o menor item que sobrou e colocando-o na ordem correta.

c. Insertion Sort (Ordenação por Inserção): Este método funciona de forma muito parecida com a maneira como se organizam cartas de baralho na mão. A estratégia é a **inserção progressiva**. O algoritmo pega um elemento por vez e olha para trás, para os itens que já ordenou, procurando o "espaço" correto para encaixar (inserir) esse novo número. Ele é muito eficiente quando os dados já estão quase todos na ordem certa.

d. Merge Sort (Ordenação por Intercalação): Este é um algoritmo mais sofisticado que usa a estratégia de **"Dividir para Conquistar"**. Primeiro, ele divide a lista original ao meio repetidas vezes até que fiquem apenas pedaços com 1 elemento. Depois, começa a fase de conquista: ele volta juntando esses pedaços, mas garantindo que, ao juntar, os números fiquem na ordem certa. Assim, ele constrói lista ordenada do zero.

e. Quick Sort (Ordenação Rápida): Também usa a estratégia de **"Dividir para Conquistar"**, mas com um foco diferente: o **particionamento**. Ele escolhe um número para ser o "pivô". A partir daí, ele reorganiza a lista jogando tudo que é menor que o pivô para a esquerda e tudo que é maior para a direita. Com o pivô no lugar certo, ele repete o processo para os dois lados que sobraram.

f. Shell Sort: O Shell Sort é basicamente uma versão turbinada do Insertion Sort. A estratégia dele é usar **incrementos (gaps) decrescentes**. Em vez de comparar apenas vizinhos, ele compara elementos que estão distantes um do outro. Isso permite que números pequenos que estão no final da lista deem "pulos" grandes para o começo rapidamente. Conforme o algoritmo roda, a distância (gap) diminui até virar 1, finalizando com um ajuste fino muito rápido.

Conclusão do Trabalho

a. Comparação entre Desempenho Teórico e Prático

Os testes realizados confirmaram exatamente o que a teoria da complexidade (Big-O) prevê. Em listas pequenas, com 10 ou 100 elementos, a diferença de tempo entre os algoritmos foi pouco perceptível. No entanto, ao utilizar listas com 10.000 números, o comportamento teórico se tornou evidente na prática.

Os algoritmos mais simples — Bubble Sort, Selection Sort e Insertion Sort — apresentaram tempos de execução muito elevados quando o tamanho da lista aumentou, demonstrando que não são adequados para lidar com grandes volumes de dados. O crescimento do tempo de execução aconteceu de forma muito rápida, como previsto para algoritmos $O(n^2)$.

Por outro lado, os algoritmos mais eficientes — Quick Sort, Merge Sort e Shell Sort — mantiveram tempos baixos mesmo com listas grandes, confirmando sua superioridade em cenários reais.

Um ponto relevante foi o desempenho do Insertion Sort. Apesar de também possuir complexidade quadrática, ele apresentou resultados superiores ao Bubble Sort e ao Selection Sort em diversos testes. Isso evidencia que, em listas pequenas ou com alto grau de ordenação prévia, o Insertion Sort se torna significativamente mais eficiente e pode ser uma escolha adequada nessas situações.

b. Qual algoritmo usar?

- **Quick Sort:** Foi o algoritmo com melhor desempenho geral nos testes. É uma ótima escolha para uso cotidiano, especialmente para listas grandes, devido à sua velocidade e eficiência na maioria das situações.
- **Merge Sort:** É uma alternativa segura ao Quick Sort. Seu desempenho é estável, mantendo $O(n \log n)$ em todos os casos, e ele ainda preserva a ordem de elementos iguais, o que pode ser necessário em algumas aplicações.
- **Shell Sort:** Obteve desempenho muito bom e ficou próximo dos algoritmos mais eficientes. Apesar de sua complexidade depender da sequência de gaps utilizada, na prática mostrou ser bastante rápido.
- **Insertion Sort:** Entre os algoritmos simples, é praticamente o único que possui aplicabilidade real. Funciona muito bem para listas pequenas ou quase ordenadas e é comum ser usado como otimização em algoritmos híbridos.
- **Bubble Sort e Selection Sort:** Apresentaram o pior desempenho prático. São algoritmos importantes do ponto de vista didático, mas raramente utilizados em sistemas reais devido ao baixo desempenho.

